Sacha

DELFERIERE

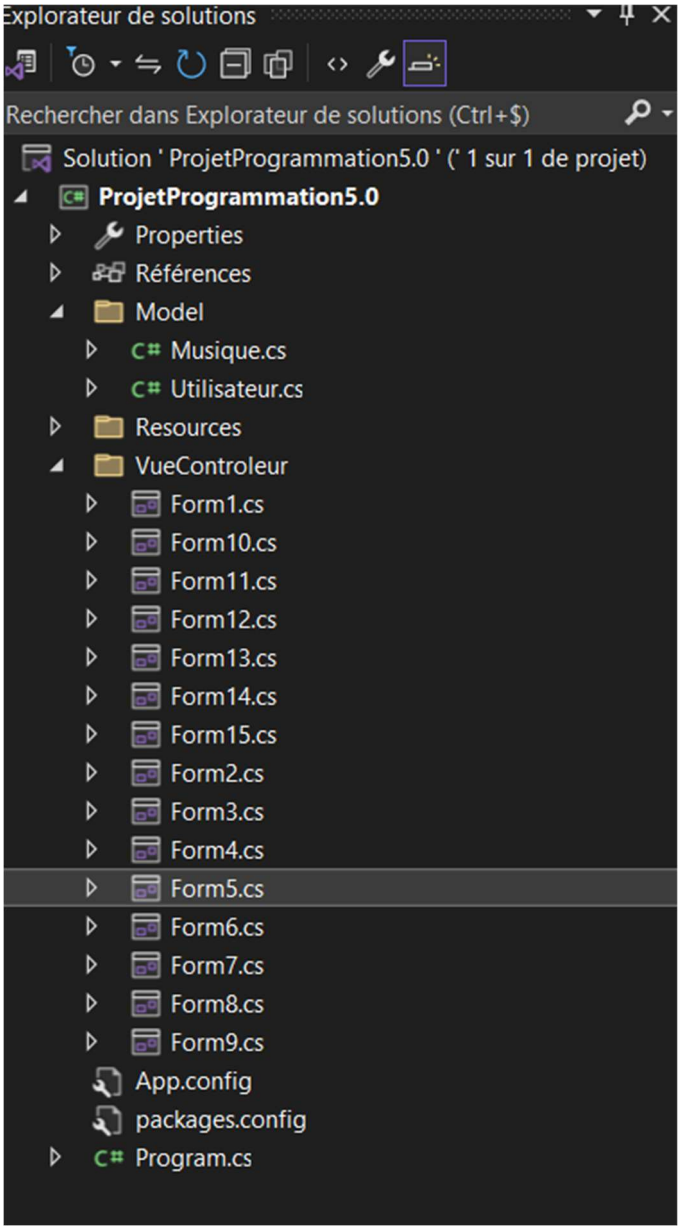
BAC 1 Q2

Rapport projet laboratoire Q2
Programmation 2023-2024

Pour ce projet j'ai voulu concevoir un jeu vidéo, aimant jouer au jeux vidéo, je me suis donc lancé dans cet univers. Je voulais tenter sur Unity ou Unreal engine, cependant, l'examen se fera sur du WindowsForm et donc je préfère m'entrainer tout en faisant mon projet. J'ai repris ci-dessous al grille de cotation pour que voyez ce que je pense avoir fait ou non. Je pense que tous sont fait mise a part l'héritage que je n'ai pas réussi à faire. J'ai essayé mais à chaque fois j'avais des problèmes entre les différentes interfaces graphiques. Je vous expliquerais plus en détails dans se rapport. Le jeu est basé sur Mario, un petit jeu ou il faut ramasser des pièces et atteindre une porte tout-en prenant la clé pour pouvoir réussir le niveau. Il faut aussi ramassez toutes les pièces pour avoir 3 étoiles, sinon il n'y aura pas toutes les étoiles et le niveau ne sera pas réussie au complet. Il y a des fonction de retour pour revenir au différents menus et pour quitter le jeu.

Utilisation des notions d'orienté objet	Oui
Découpe du code avec la notion de MVC	Oui
Utilisation d'une interface graphique	Oui
Utilisation de l'héritage pertinent et cohérente	Non
Propreté et qualité du code	Oui
Importance du projet	Oui
Complétude du produit fini	Oui
Exactitude du rapport	?
Présentation orale	?
Programme fonctionnel	Oui

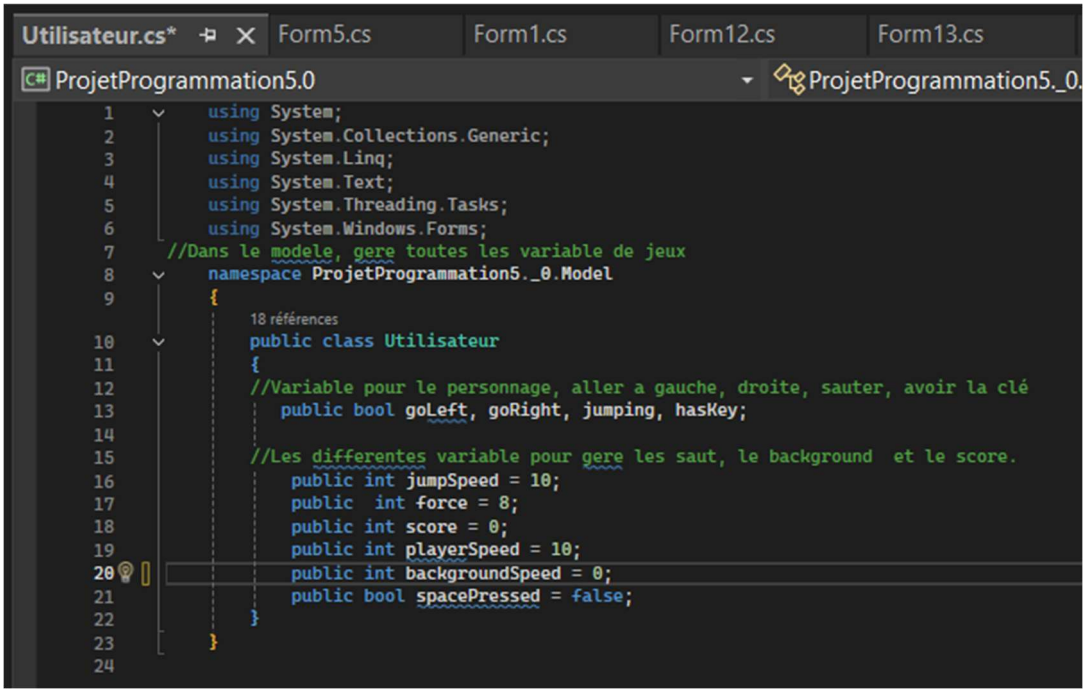
Dans se code il y a un dossier (Vue et contrôleur) contenant 15 Forms (interfaces graphique), ainsi qu’un et un dossier model contenant deux classes. Et le prorgrams.cs.



Le model est bien séparer de la vue et du contrôleur, la vue et le contrôleur sont les interfaces graphique la vue le form.designer.cs et le contrôleur le form.cs.

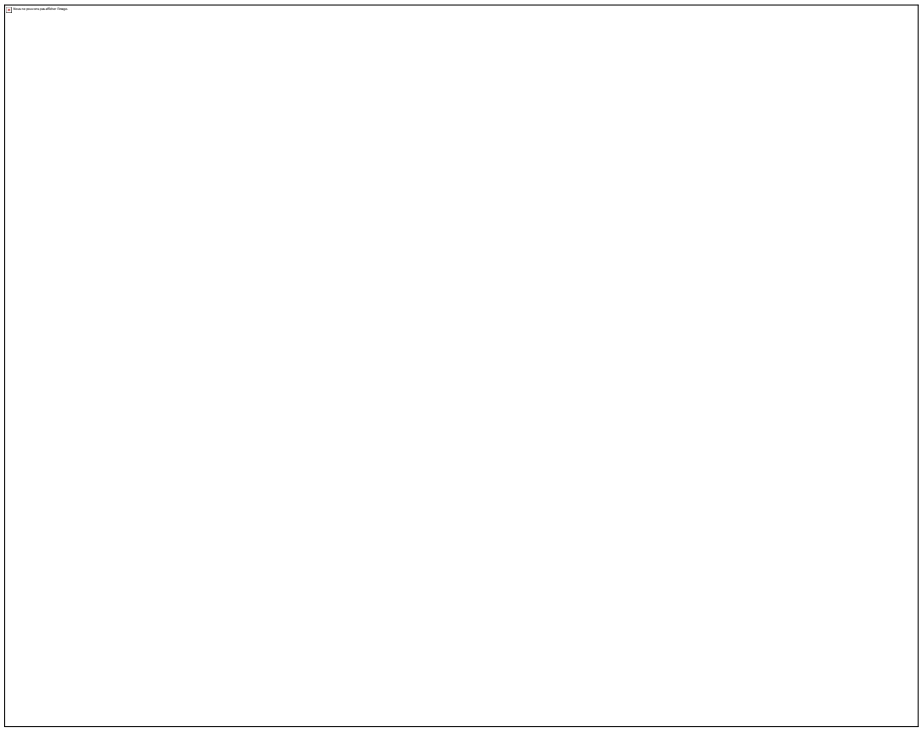
Il y a deux model, le premier Utilisateur. Il sert pour tous le code, les variables les plus utiliser sont stoker là. Surtout pour le personnage et ses mouvements, mais aussi le « backgroundSpeed » qui était censé faire bouger le fond pour faire comme-ci le personnage avancer. Je l'ai laissé dans le code mais le rendu graphique est vraiment pas beau et donc j'ai décidé de le « désactivé ». Si vous voulez l'utiliser vous pouvez en mettant la variable « backgroundSpeed» avec la valeur 8.

Model, Utilisateur :

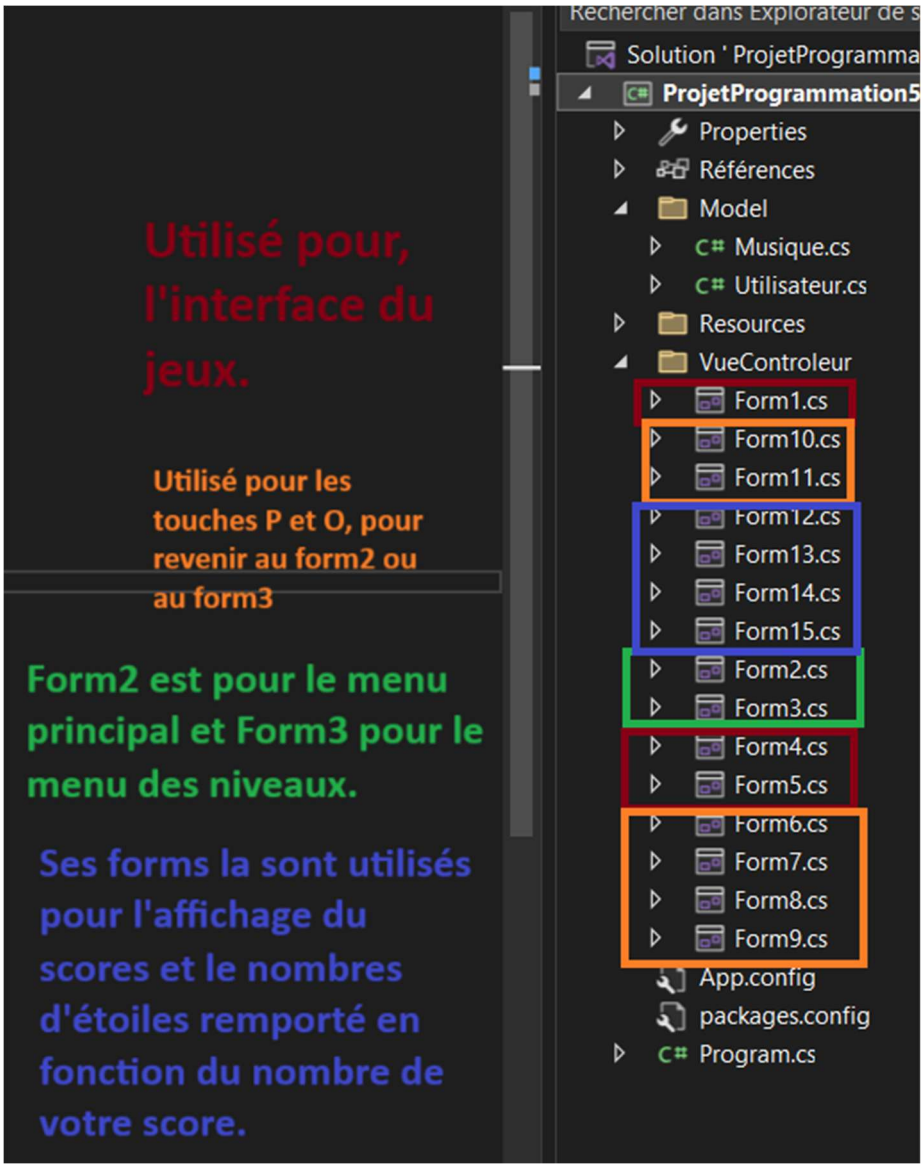


Ensuite le modele Musique, lui il est utilise pour la musique du jeu. Il est activé dans le form2 et se lance dés que l'application est lancé. J'ai créé cette classe parce que sinon la musique des piéce et celle de la musique du form3 ne fonctionner pas ensemble. Donc j'ai du installé un packages NuGet nommé, NAudio.Wave et regarder sur interent comment l'utiliser.

Model, Musique :

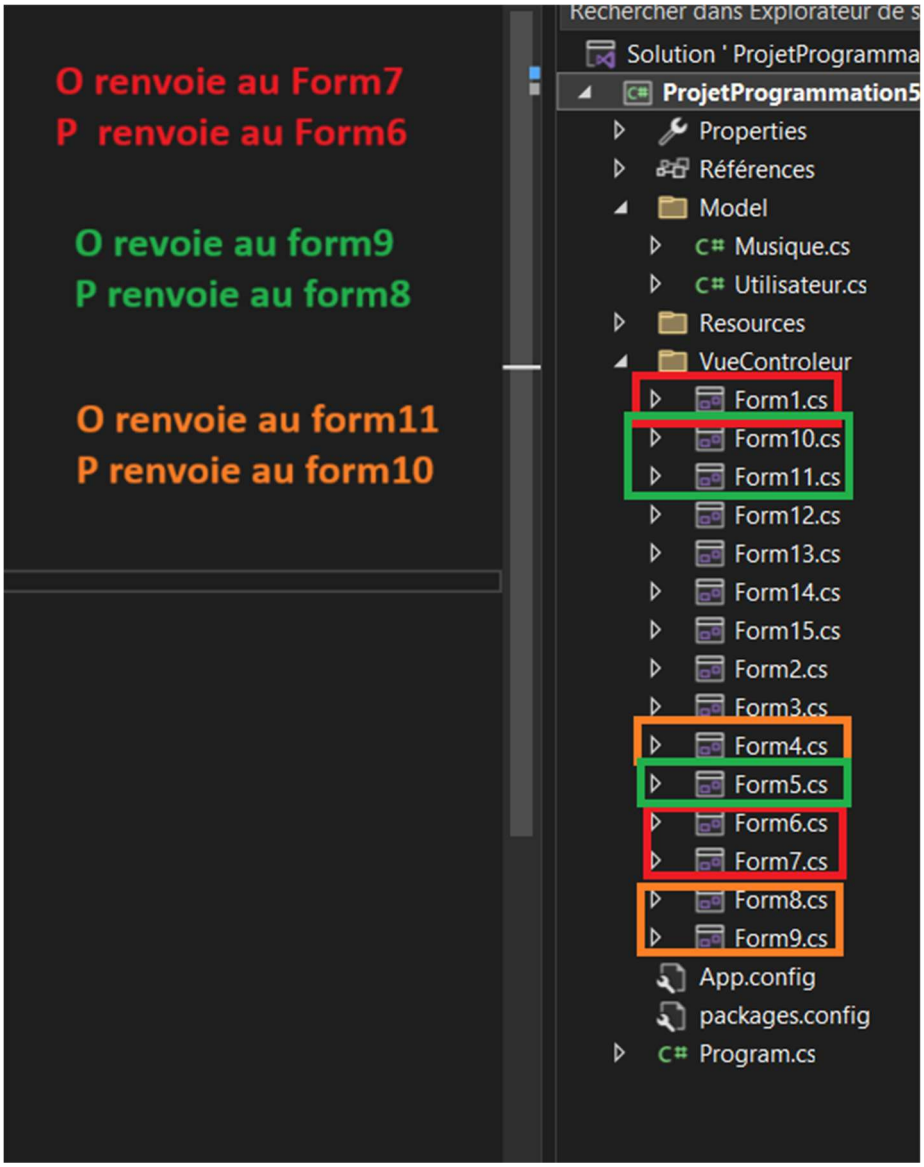


Voila c'est tous pour les model. Passons aux interfaces graphiques et oui il y en a beaucoup (15 au total). Cependant il en existe plusieurs sorte bien spécifique pour certain cas.



Donc voilà, et donc ses Forms se ressemble beaucoup les uns des autres mais ont tous un code différent.

Pour le Form1, il fait tout le système du jeu en lui-même, comme le form4 et 5. Les Form4 et 5 sont exactement pareil, mais par exemple pour le form3, il y a des pièges en plus et le 4 des pièces en plus. Mais surtout, chacun d'eux a sont propres Form, pour le P et le O. Parce qu'enfaite je n'arrivais pas à ouvrir un Form et fermer les deux autres en même temps, et donc j'ai du faire des Forms en plus pour ne pas avoir de problèmes, enfaite c'était le plus compliquer de se code.



Le code pour ça du Form1(je vais seulement prendre les exemple du code du Form1, parce que comme je l'ai dit se sont les même mécanisme).

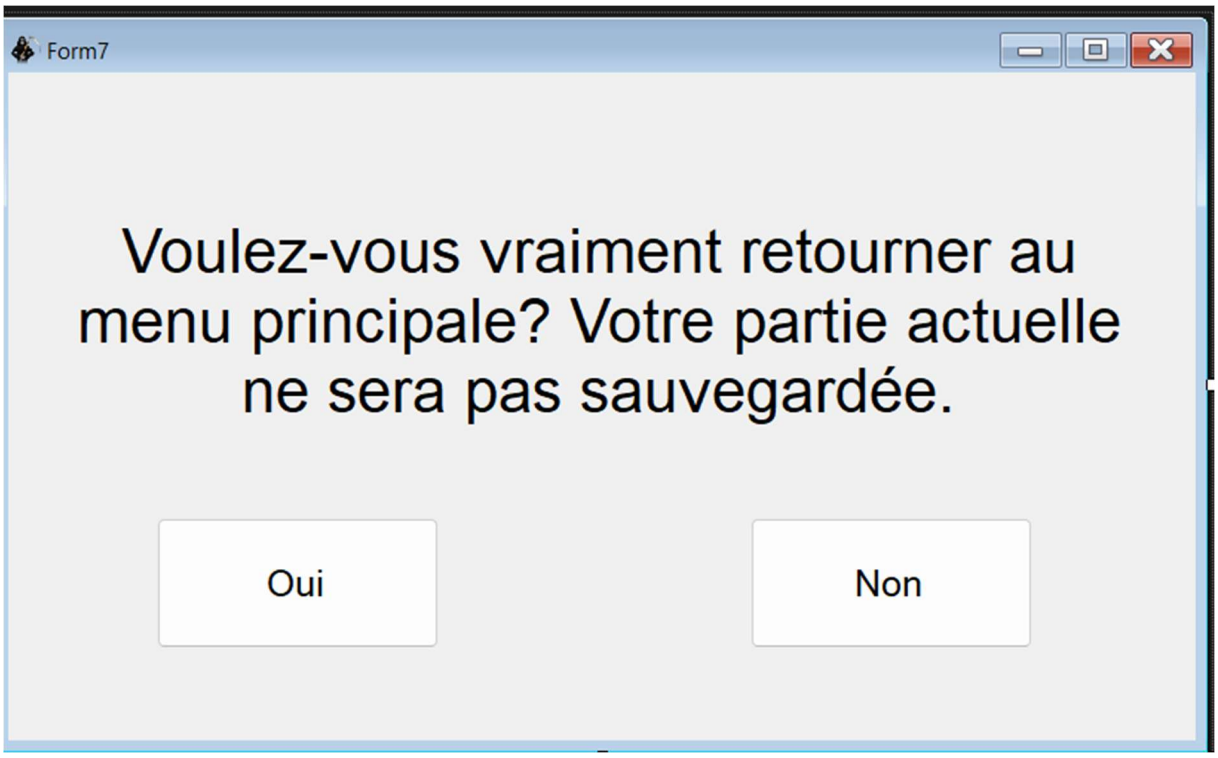
Code Form1 :

```
if (e.KeyCode == Keys.O)
{
    if (!form7Instance.Visible)
    {
        form7Instance.Show();
    }
    this.Hide();

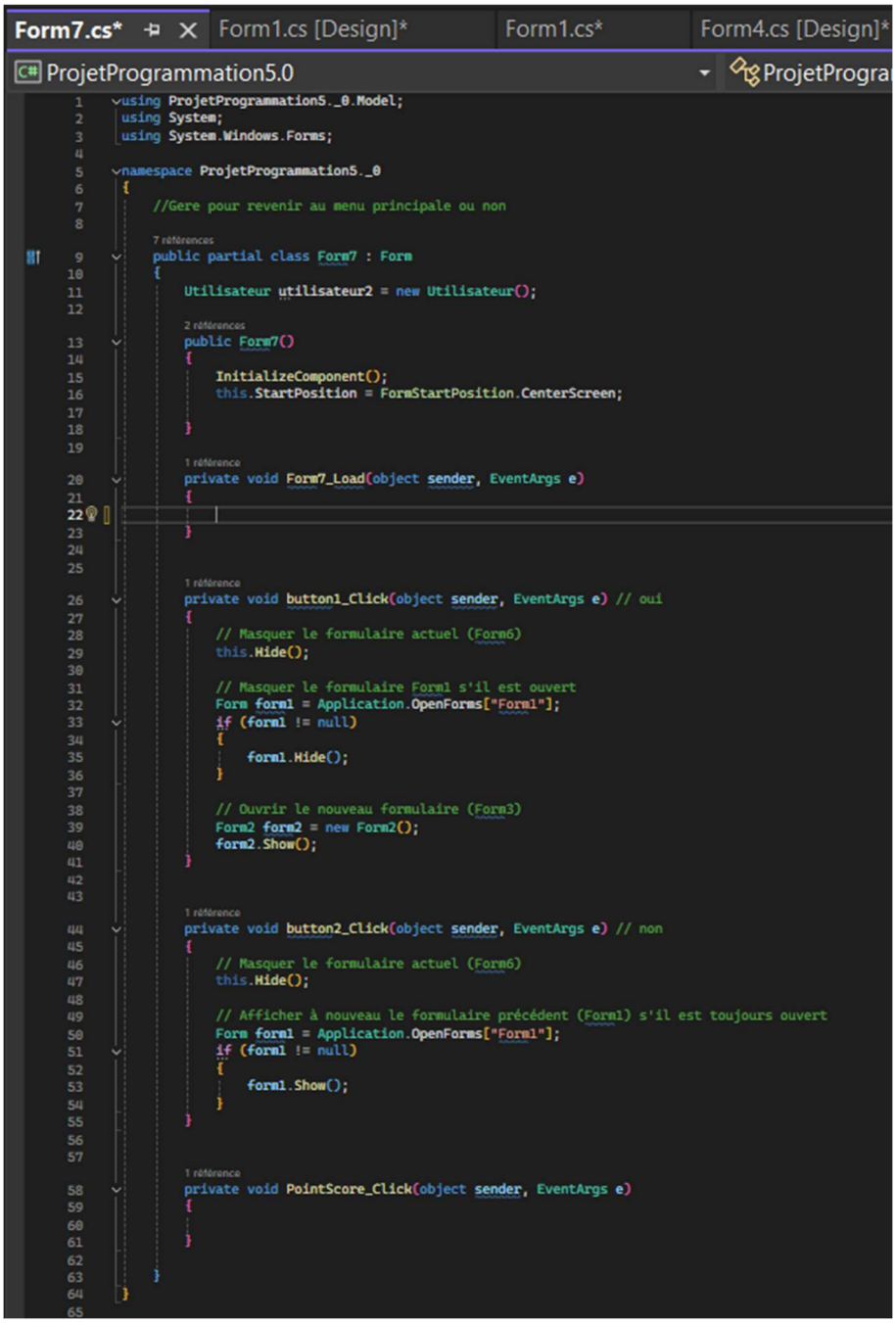
}
if (e.KeyCode == Keys.P)
{
    if (!form6Instance.Visible)
    {
        form6Instance.Show();
    }
    this.Hide();
}
```

Ceux qu’il renvoie et leurs codes.

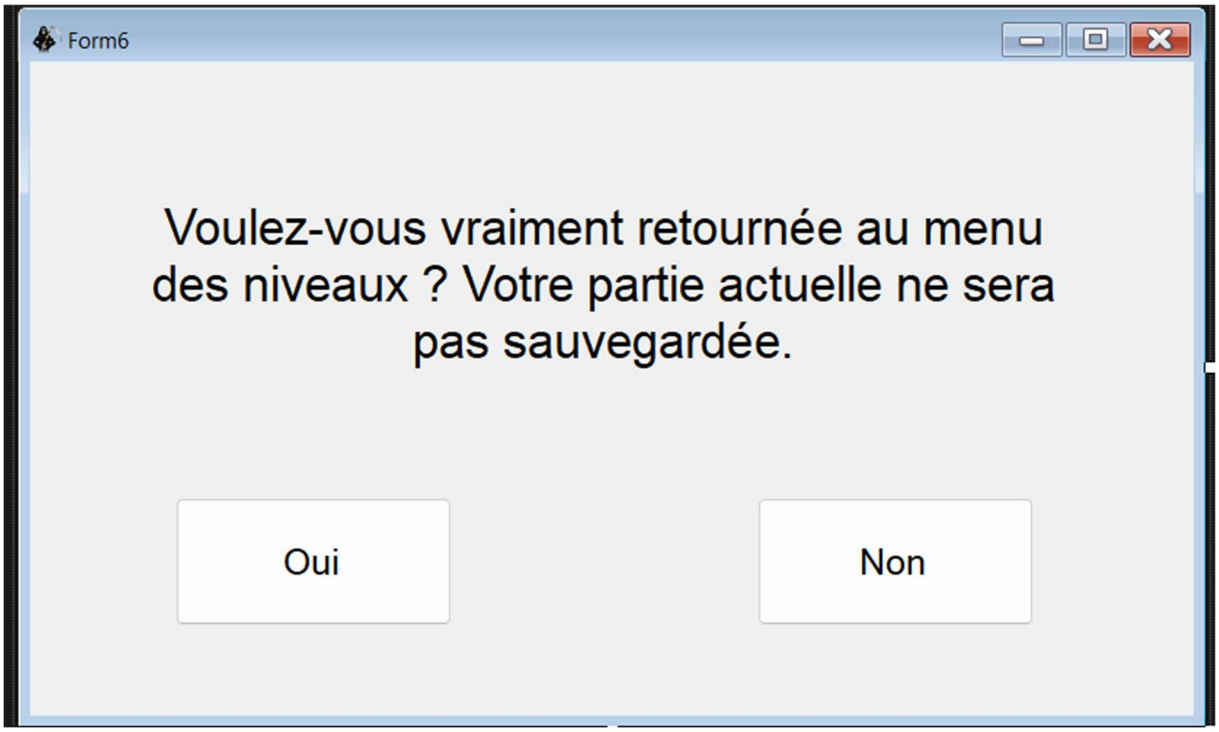
Lorsque O est pressé :



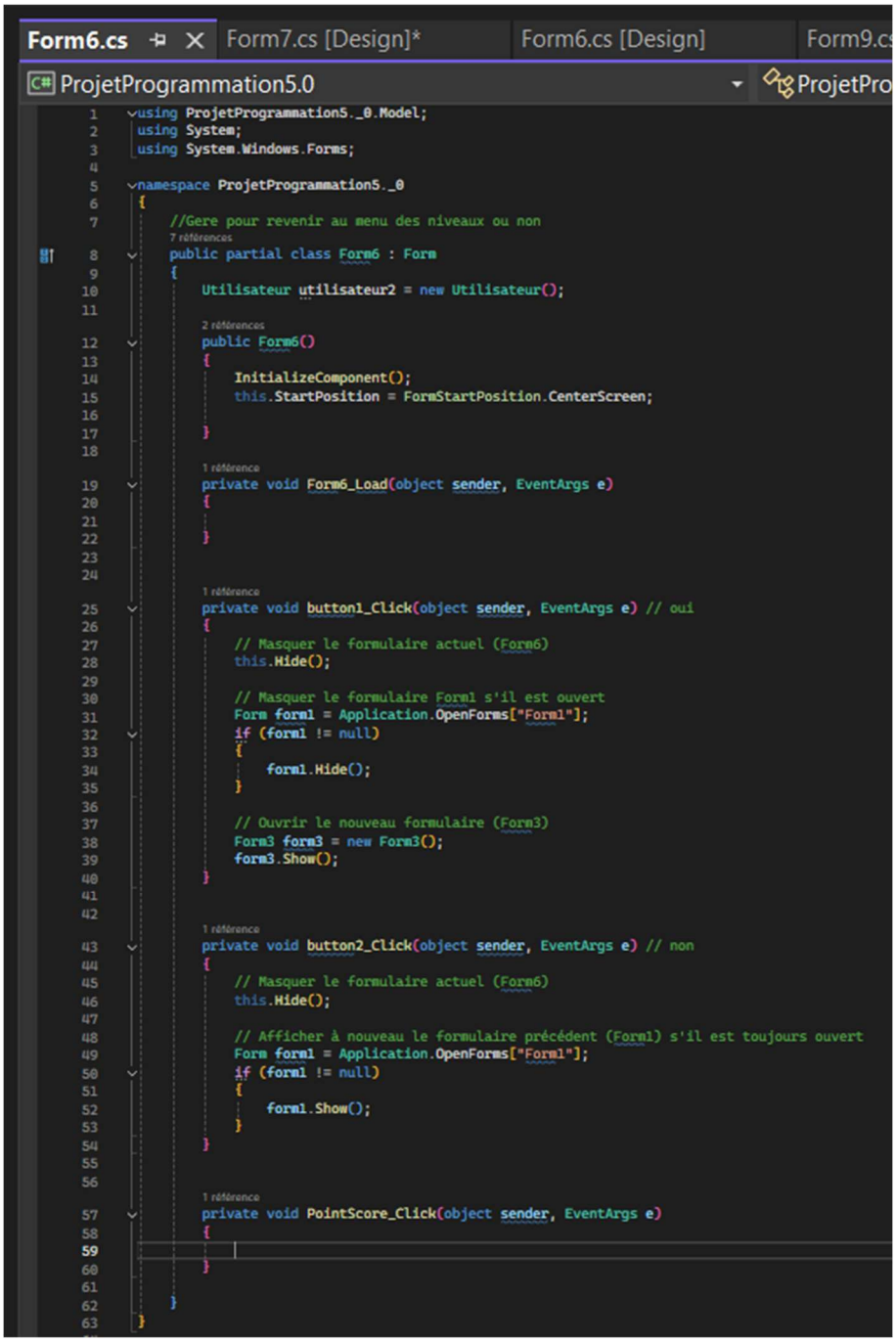
Code form 7 :



Lorsque P est pressé :


Form6
Voulez-vous vraiment retournée au menu des niveaux ? Votre partie actuelle ne sera pas sauvegardée.
Oui
Non

Code Form6 :




```
using ProjetProgrammation5._0.Model;
using System;
using System.Windows.Forms;

namespace ProjetProgrammation5._0
{
    //Gere pour revenir au menu des niveaux ou non
    7 références
    public partial class Form6 : Form
    {
        Utilisateur utilisateur2 = new Utilisateur();

        2 références
        public Form6()
        {
            InitializeComponent();
            this.StartPosition = FormStartPosition.CenterScreen;

        }

        1 référence
        private void Form6_Load(object sender, EventArgs e)
        {

        }


        1 référence
        private void button1_Click(object sender, EventArgs e) // oui
        {
            // Masquer le formulaire actuel (Form6)
            this.Hide();

            // Masquer le formulaire Form1 s'il est ouvert
            Form form1 = Application.OpenForms["Form1"];
            if (form1 != null)
            {
                form1.Hide();
            }

            // Ouvrir le nouveau formulaire (Form3)
            Form3 form3 = new Form3();
            form3.Show();
        }


        1 référence
        private void button2_Click(object sender, EventArgs e) // non
        {
            // Masquer le formulaire actuel (Form6)
            this.Hide();

            // Afficher à nouveau le formulaire précédent (Form1) s'il est toujours ouvert
            Form form1 = Application.OpenForms["Form1"];
            if (form1 != null)
            {
                form1.Show();
            }
        }

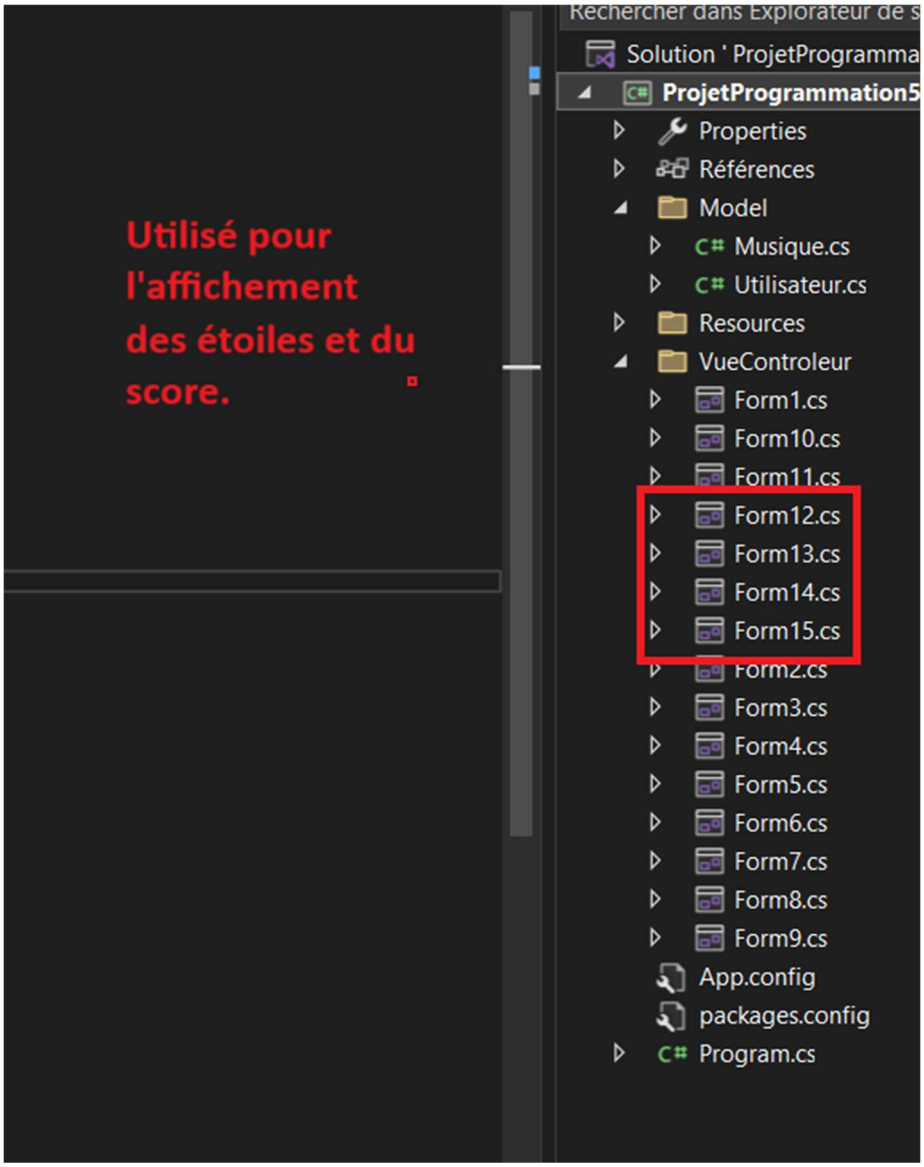

        1 référence
        private void PointScore_Click(object sender, EventArgs e)
        {

        }

    }
}
```

Et c’est pareil pour les autres Forms comme je l’ai expliqué au-dessus.

Ensuite il existe les Forms de 12 à 15 compris qui sont utilisé pour l’affichement des étoiles et du score.



Affichement et leurs codes (j'en mettrais qu'un code parce se sont exactement les mêmes codes).

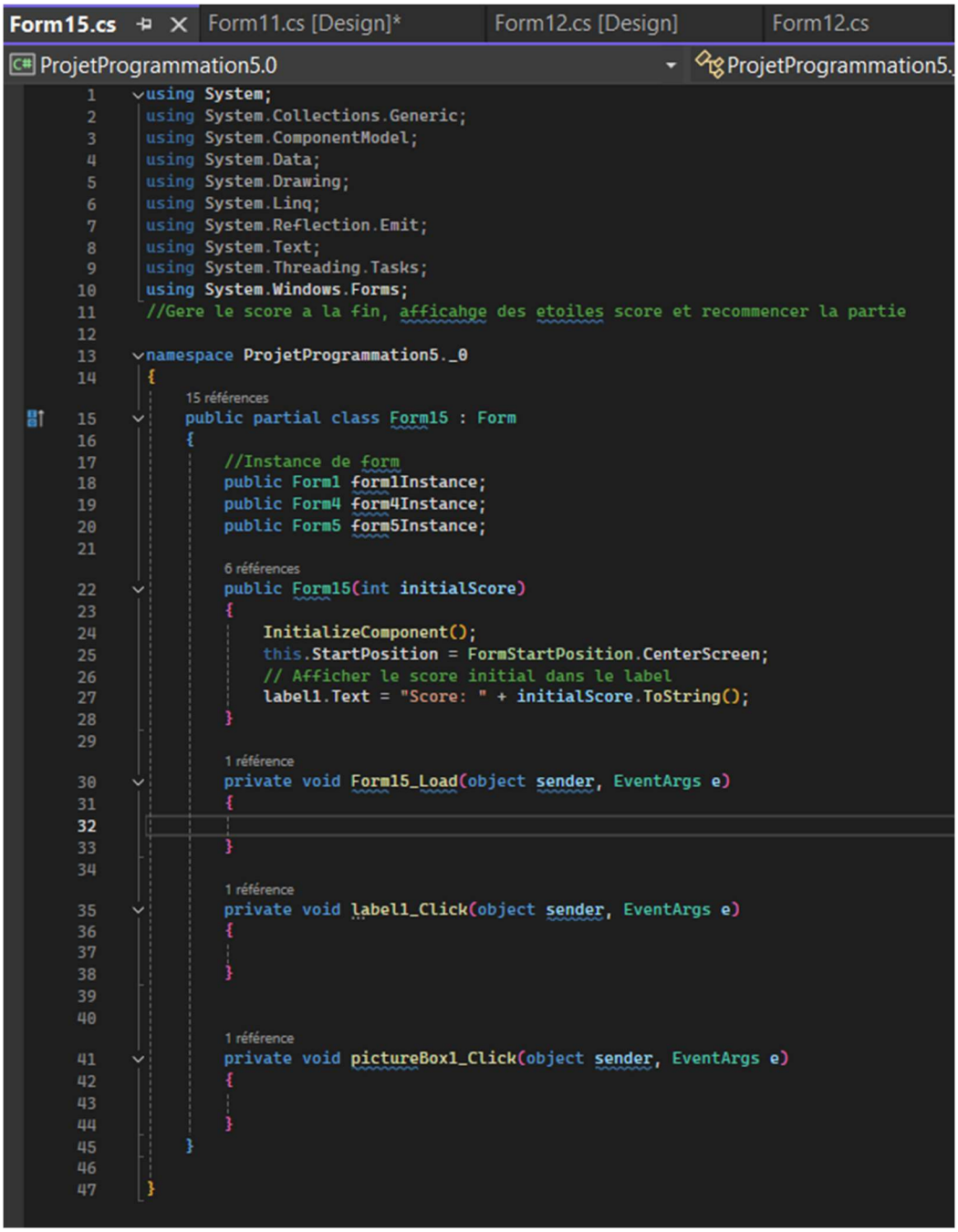

Form12
Score : 0
T'es vraiment nul...


Form12
Score : 0
Tu peux faire mieux...





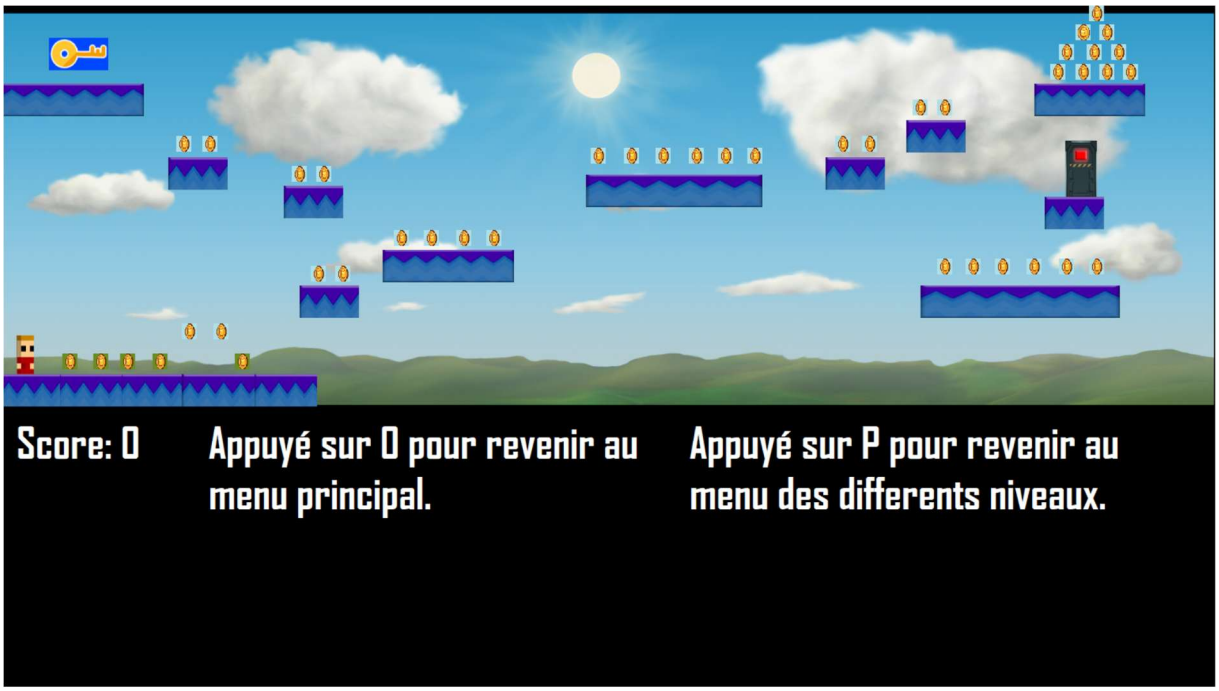
Code du Form15, exactement le même que les Forms 12,13,14.



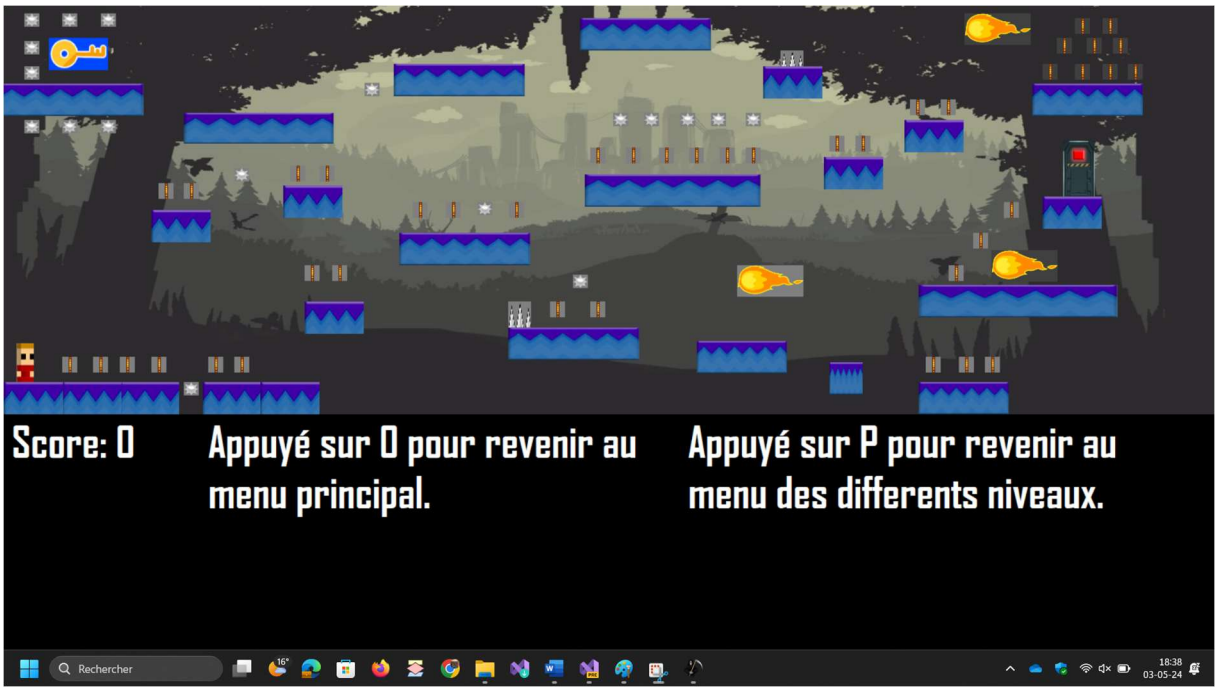
Voilà pour les différentes présentations des différents codes et de leurs utilités ainsi que leurs interfaces. Maintenant je vais vous présentez le code qui gère tous ça et qui switch de Form a Form. Donc il en existe 3 : le Form1, Form4 et Form5. Tous sont pareil, la seule différence réside dans les pièges et potentiellement quelques noms qui sont changé suite des modifications. Je mettrais les choses ajoutées en plus dans les autres Forms.

Je vais vous montrez les différentes interfaces graphiques.

Form1 :



Form2 :

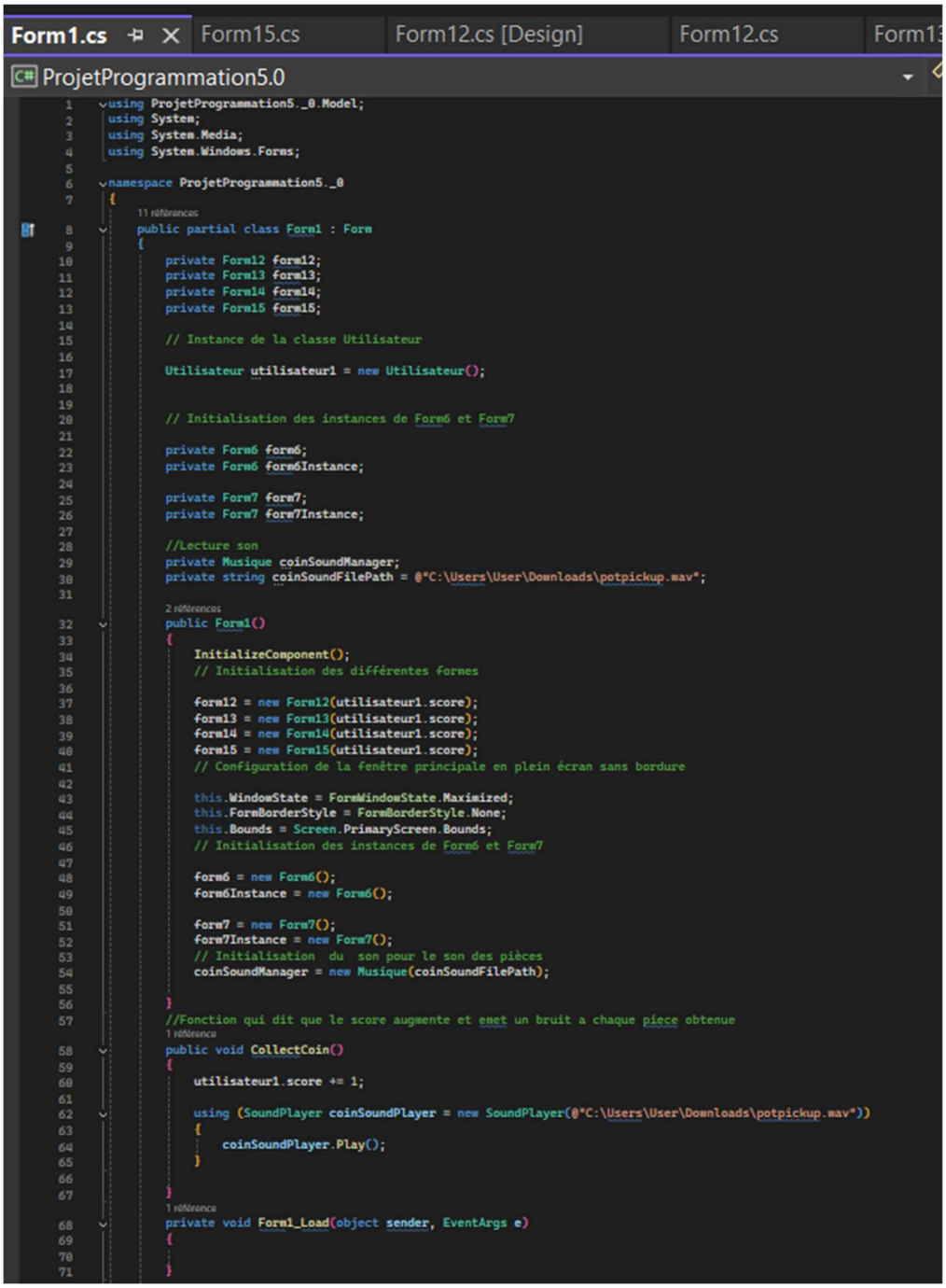


Form3 :



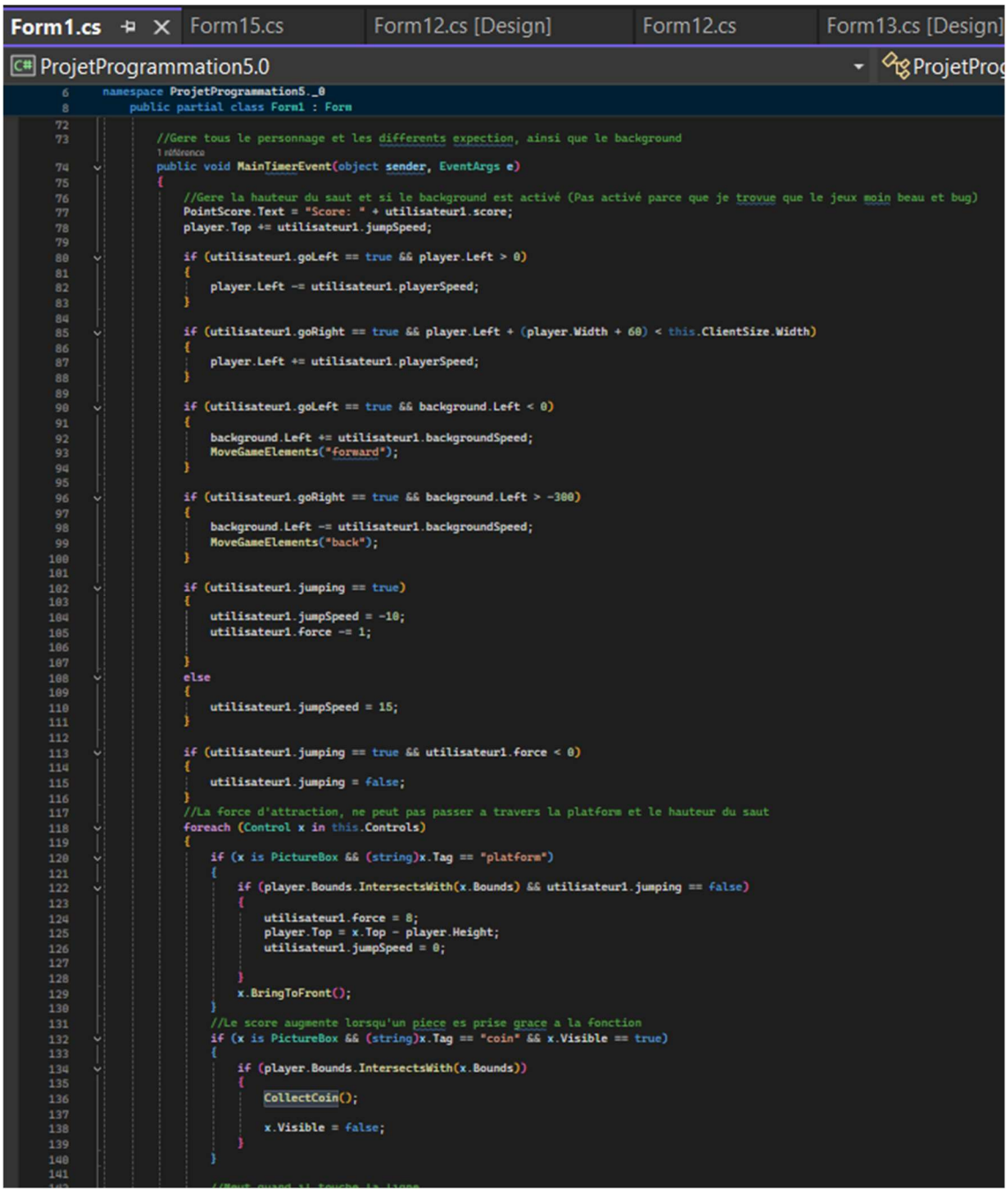
Appart leur interface graphique le mécanisme du code et même le code reste le même pour chacun d'entre eux.

Donc voila le code.



Dans cette première parti du code, il y a la configuration de la fenêtre avec des plusieurs initialisations d'autres Forms qui seront utilisés plus tard.

La fonction CollectCoin est utilisé pour ajouter le score qui augmente a chaque pièce prises et a joué le son dés qu'une pièce est prise par le personnage. J'ai bien galéré a trouver comme faire fonctionne deux sons en même temps, parce il y a le son du début qui fonctionne aussi.




```
namespace ProjetProgrammation5._0
    public partial class Form1 : Form

        //Gere tous le personnage et les differents expection, ainsi que le background
        1 référence
        public void MainTimerEvent(object sender, EventArgs e)
        {
            //Gere la hauteur du saut et si le background est activé (Pas activé parce que je trovue que le jeux moin beau et bug)
            PointScore.Text = "Score: " + utilisateur1.score;
            player.Top += utilisateur1.jumpSpeed;

            if (utilisateur1.goLeft == true && player.Left > 0)
            {
                player.Left -= utilisateur1.playerSpeed;
            }

            if (utilisateur1.goRight == true && player.Left + (player.Width + 60) < this.ClientSize.Width)
            {
                player.Left += utilisateur1.playerSpeed;
            }

            if (utilisateur1.goLeft == true && background.Left < 0)
            {
                background.Left += utilisateur1.backgroundSpeed;
                MoveGameElements("forward");
            }

            if (utilisateur1.goRight == true && background.Left > -300)
            {
                background.Left -= utilisateur1.backgroundSpeed;
                MoveGameElements("back");
            }

            if (utilisateur1.jumping == true)
            {
                utilisateur1.jumpSpeed = -10;
                utilisateur1.force -= 1;

            }
            else
            {
                utilisateur1.jumpSpeed = 15;
            }

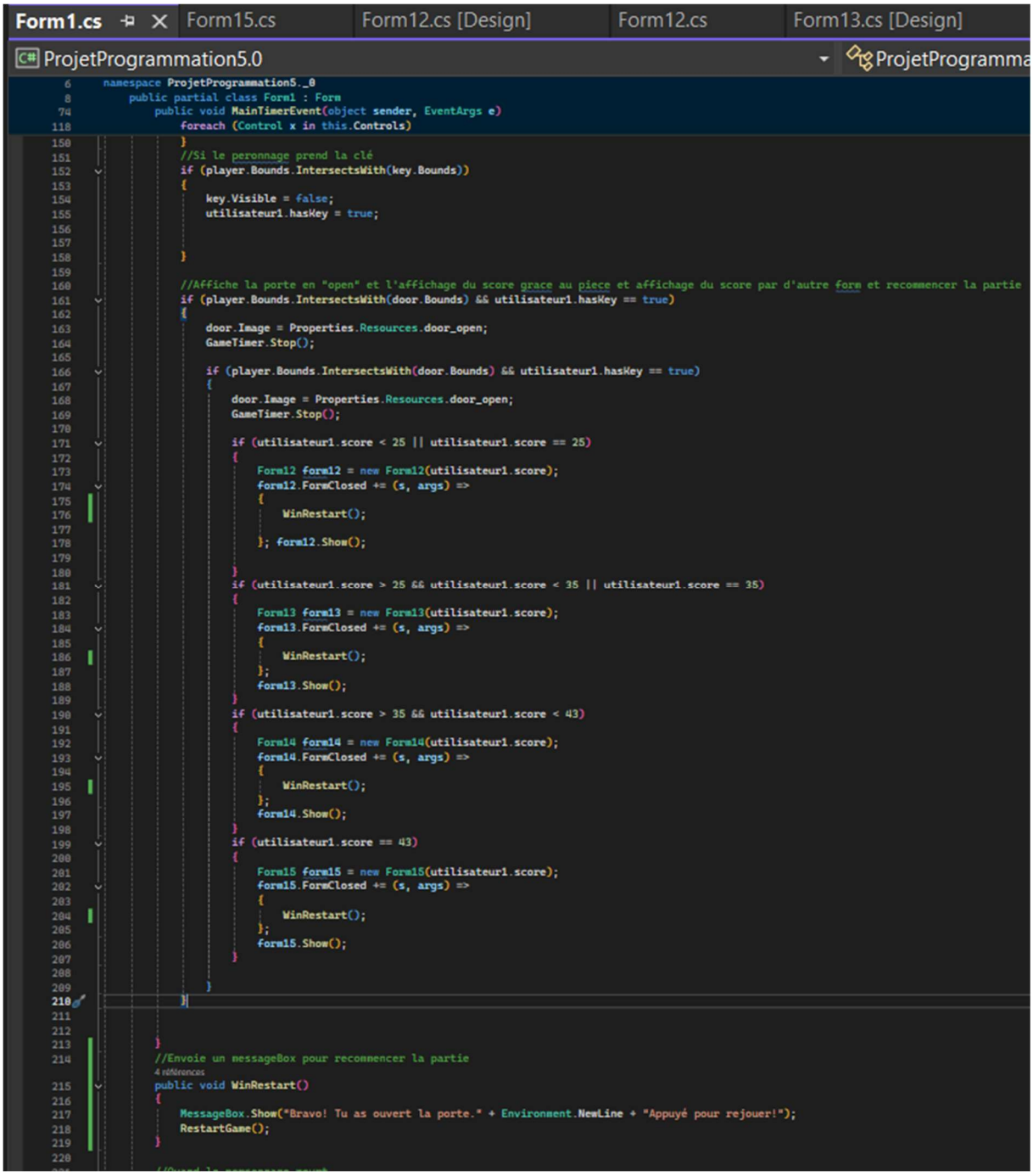
            if (utilisateur1.jumping == true && utilisateur1.force < 0)
            {
                utilisateur1.jumping = false;
            }
            //La force d'attraction, ne peut pas passer a travers la platform et le hauteur du saut
            foreach (Control x in this.Controls)
            {
                if (x is PictureBox && (string)x.Tag == "platform")
                {
                    if (player.Bounds.IntersectsWith(x.Bounds) && utilisateur1.jumping == false)
                    {
                        utilisateur1.force = 8;
                        player.Top = x.Top - player.Height;
                        utilisateur1.jumpSpeed = 0;

                    }
                    x.BringToFront();
                }
                //Le score augmente lorsqu'un piece es prise grace a la fonction
                if (x is PictureBox && (string)x.Tag == "coin" && x.Visible == true)
                {
                    if (player.Bounds.IntersectsWith(x.Bounds))
                    {
                        CollectCoin();

                        x.Visible = false;
                    }
                }
```

Ici nous avons tous le main, enfaite tous est gérer la et les fonctions y sont appelé la aussi.

La première partie gère les sauts, les déplacement du personnages, la force de la gravité et le background qui ne sera pas utilisé. Ensuite ceux qui gère les platform et les pièces et donc c'est ici qu'est appelé la fonction CollectCoin.



```
Form1.cs | Form15.cs | Form12.cs [Design] | Form12.cs | Form13.cs [Design]
ProjetProgrammation5.0
```

```
namespace ProjetProgrammation5._0
    public partial class Form1 : Form
        public void MainTimerEvent(object sender, EventArgs e)
            foreach (Control x in this.Controls)
            }
            //Si le peronnage prend la clé
            if (player.Bounds.IntersectsWith(key.Bounds))
            {
                key.Visible = false;
                utilisateur1.hasKey = true;


            }

            //Affiche la porte en "open" et l'affichage du score grace au piece et affichage du score par d'autre form et recommencer la partie
            if (player.Bounds.IntersectsWith(door.Bounds) && utilisateur1.hasKey == true)
            {
                door.Image = Properties.Resources.door_open;
                GameTimer.Stop();

                if (player.Bounds.IntersectsWith(door.Bounds) && utilisateur1.hasKey == true)
                {
                    door.Image = Properties.Resources.door_open;
                    GameTimer.Stop();

                    if (utilisateur1.score < 25 || utilisateur1.score == 25)
                    {
                        Form12 form12 = new Form12(utilisateur1.score);
                        form12.FormClosed += (s, args) =>
                        {
                            WinRestart();

                        }; form12.Show();

                    }
                    if (utilisateur1.score > 25 && utilisateur1.score < 35 || utilisateur1.score == 35)
                    {
                        Form13 form13 = new Form13(utilisateur1.score);
                        form13.FormClosed += (s, args) =>
                        {
                            WinRestart();
                        };
                        form13.Show();
                    }
                    if (utilisateur1.score > 35 && utilisateur1.score < 43)
                    {
                        Form14 form14 = new Form14(utilisateur1.score);
                        form14.FormClosed += (s, args) =>
                        {
                            WinRestart();
                        };
                        form14.Show();
                    }
                    if (utilisateur1.score == 43)
                    {
                        Form15 form15 = new Form15(utilisateur1.score);
                        form15.FormClosed += (s, args) =>
                        {
                            WinRestart();
                        };
                        form15.Show();
                    }

                }
            }

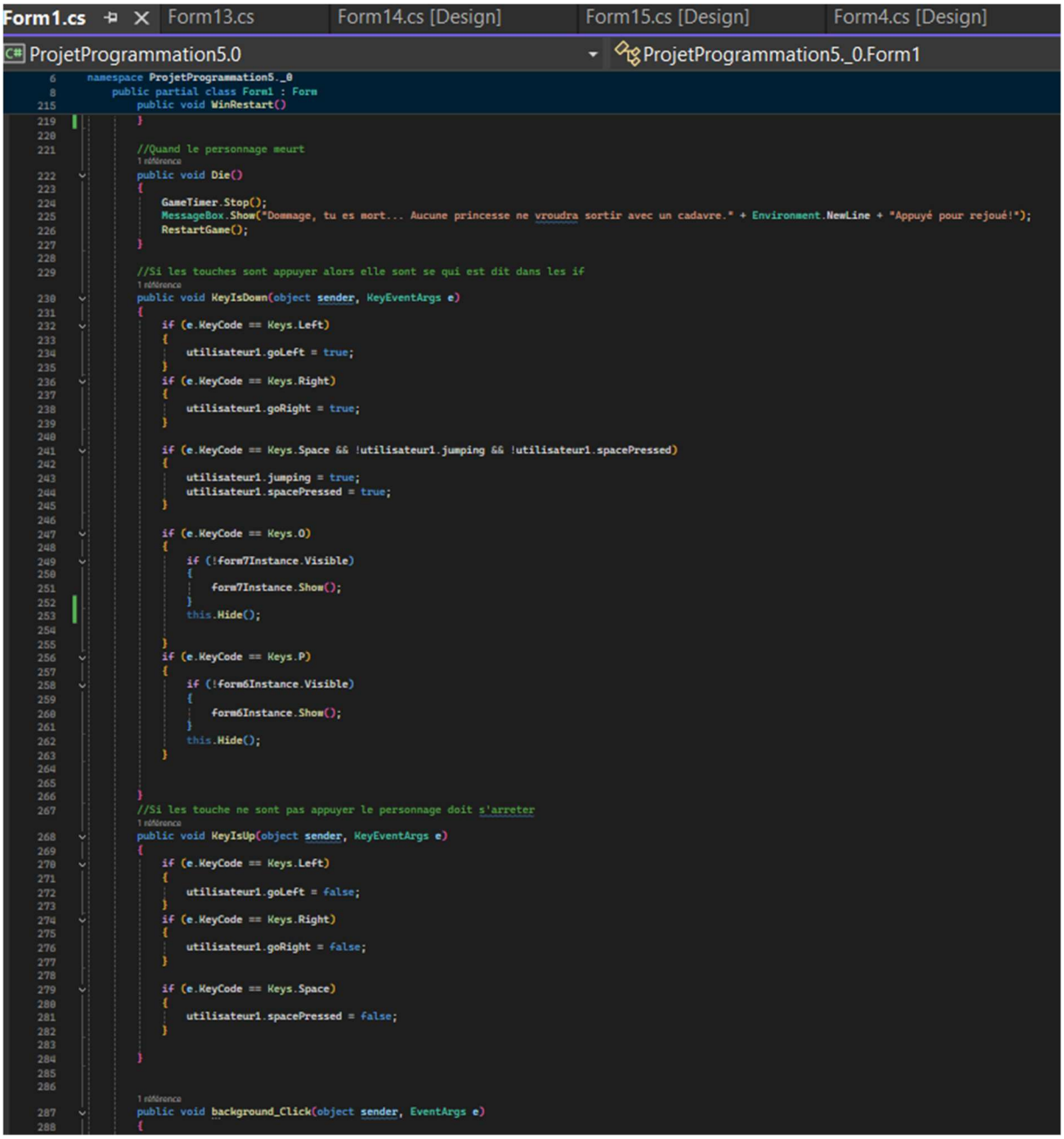

        }
        //Envoie un messageBox pour recommencer la partie
        4 références
        public void WinRestart()
        {
            MessageBox.Show("Bravo! Tu as ouvert la porte." + Environment.NewLine + "Appuyé pour rejouer!");
            RestartGame();
        }
```

Puis nous avons, encore des « if » qui gère toutes les exceptions. Donc le premier pour dire si la clé est prise ou non. Et ensuite C'est ici que les Forms sont utilisés pour dire que si la clé est dans la porte, la porte change de photo et l'affichage d'un Form suivant le nombre du score.

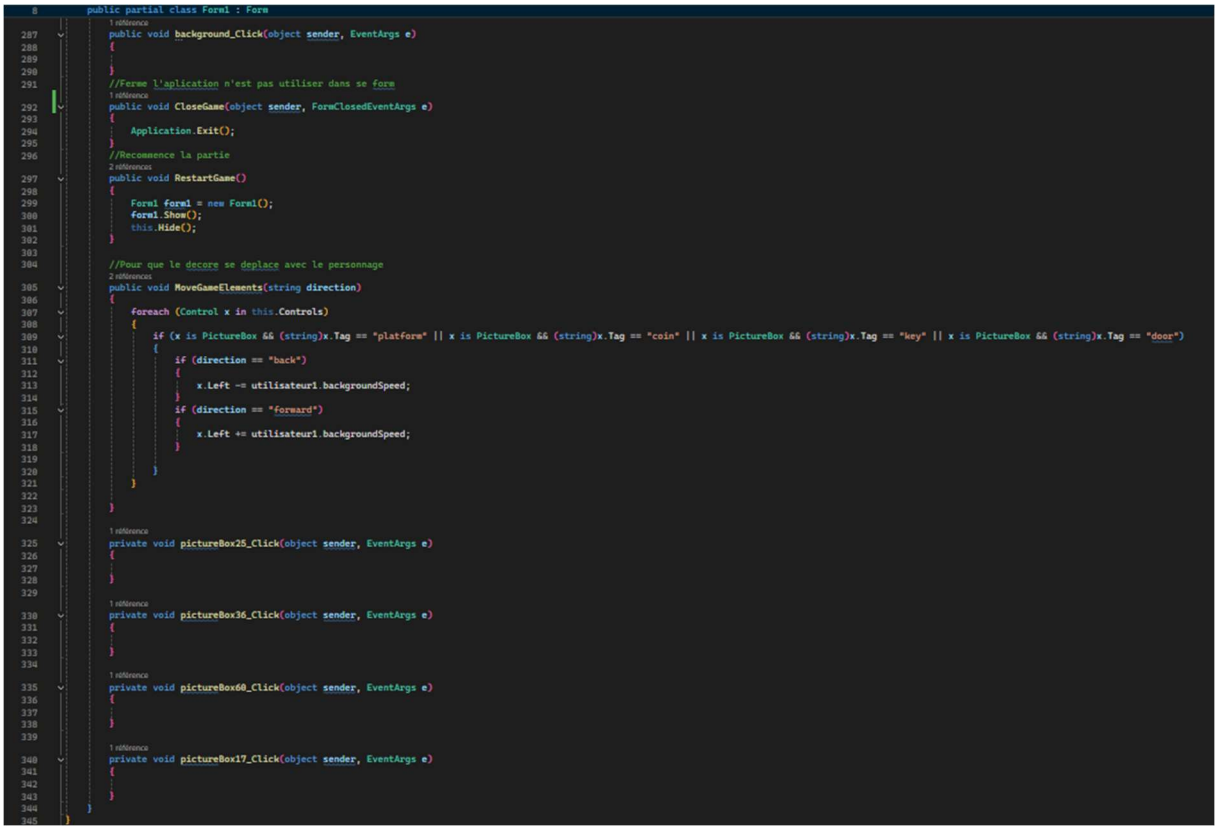
Et la fonction WinRestart envoie un messageBox et reprend la fonctione RestartGame plus bas.

```
if (x is PictureBox && (string)x.Tag == "LigneTomber" && x.Visible == true)
{
    if (player.Bounds.IntersectsWith(x.Bounds))
    {
        Die();
    }
}
```

Je l'ai oublié mais c'est pour dire que une fois la limite vers le bas est touché alors le personnage meurs.



La fonction Die est utilisée pour afficher un message quand on meurt et recommencer une partie. KeyIsDown et KeyIsUp sont utiliser pour les touches en bas à gauche du clavier. J'aurais pu très bien mettre z,q,s,d mais souvent les jeux de se genre sont joué sur le petit clavier de flèches. KeyIsUp c'est quand la touche est appuyée et down quand elle ne l'est pas. Donc lorsque la touche est relâchée le personnage s'arrête.



```
public partial class Form1 : Form
    1 référence
    public void background_Click(object sender, EventArgs e)
    {

    }
    //Ferme l'aplication n'est pas utiliser dans se form
    1 référence
    public void CloseGame(object sender, FormClosedEventArgs e)
    {
        Application.Exit();
    }
    //Recommence la partie
    2 références
    public void RestartGame()
    {
        Form1 form1 = new Form1();
        form1.Show();
        this.Hide();
    }

    //Pour que le decore se deplace avec le personnage
    2 références
    public void MoveGameElements(string direction)
    {
        foreach (Control x in this.Controls)
        {
            if (x is PictureBox && (string)x.Tag == "platform" || x is PictureBox && (string)x.Tag == "coin" || x is PictureBox && (string)x.Tag == "key" || x is PictureBox && (string)x.Tag == "door")
            {
                if (direction == "back")
                {
                    x.Left -= utilisateur1.backgroundSpeed;
                }
                if (direction == "forward")
                {
                    x.Left += utilisateur1.backgroundSpeed;
                }

            }
        }

    }

    1 référence
    private void pictureBox25_Click(object sender, EventArgs e)
    {

    }

    1 référence
    private void pictureBox36_Click(object sender, EventArgs e)
    {

    }

    1 référence
    private void pictureBox60_Click(object sender, EventArgs e)
    {

    }

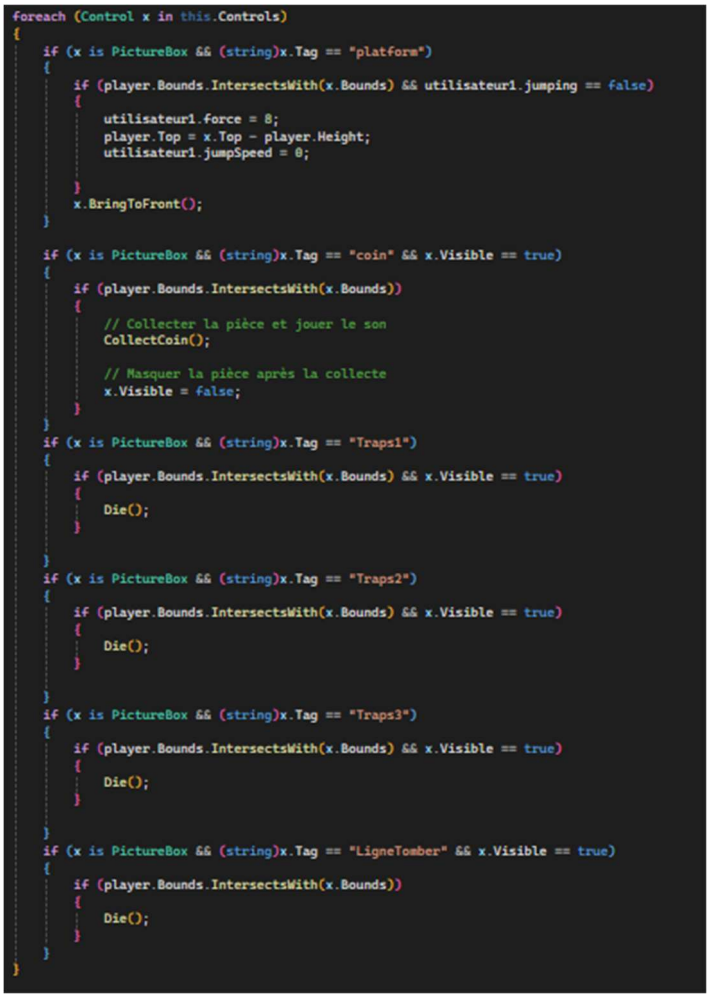
    1 référence
    private void pictureBox17_Click(object sender, EventArgs e)
    {

    }
}
```

Pour terminer, le CloseGame mais que je n'utiliserais pas dans ce form, ensuite la fameuse fonction RestartGame, donc ça réaffiche le Form1 a chaque fois que la fonction est appelé. Et le MooveElement qui est utilisé pour le background, cependant je ne l'utilise pas car elle ne fonctionne pas bien. Mais si vous voulez essayer il faut juste aller dans utilisateur et remplacer le 0 de « backgroundSpeed » en 8.

Les différentes façons dont les pièges sont utilisé, parce que dans se form il n'y en a pas.

Exemple du Form4



```
foreach (Control x in this.Controls)
{
    if (x is PictureBox && (string)x.Tag == "platform")
    {
        if (player.Bounds.IntersectsWith(x.Bounds) && utilisateur1.jumping == false)
        {
            utilisateur1.force = 8;
            player.Top = x.Top - player.Height;
            utilisateur1.jumpSpeed = 0;

        }
        x.BringToFront();
    }

    if (x is PictureBox && (string)x.Tag == "coin" && x.Visible == true)
    {
        if (player.Bounds.IntersectsWith(x.Bounds))
        {
            // Collecter la pièce et jouer le son
            CollectCoin();

            // Masquer la pièce après la collecte
            x.Visible = false;
        }
    }
    if (x is PictureBox && (string)x.Tag == "Traps1")
    {
        if (player.Bounds.IntersectsWith(x.Bounds) && x.Visible == true)
        {
            Die();
        }

    }
    if (x is PictureBox && (string)x.Tag == "Traps2")
    {
        if (player.Bounds.IntersectsWith(x.Bounds) && x.Visible == true)
        {
            Die();
        }

    }
    if (x is PictureBox && (string)x.Tag == "Traps3")
    {
        if (player.Bounds.IntersectsWith(x.Bounds) && x.Visible == true)
        {
            Die();
        }

    }
    if (x is PictureBox && (string)x.Tag == "LigneTomber" && x.Visible == true)
    {
        if (player.Bounds.IntersectsWith(x.Bounds))
        {
            Die();
        }
    }
}
```

Donc voilà, c’est le même procéder que pour la pièce ou la platform. Sauf que la c’est la fin de la partie quand un pièges est touché.

```
        }
        if (player.Bounds.IntersectsWith(door.Bounds) && utilisateur1.hasKey == true)
        {
            door.Image = Properties.Resources.door_open;
            GameTimer.Stop();

            if (player.Bounds.IntersectsWith(door.Bounds) && utilisateur1.hasKey == true)
            {
                door.Image = Properties.Resources.door_open;
                GameTimer.Stop();

                if (utilisateur1.score < 35 || utilisateur1.score == 35)
                {
                    Form12 form12 = new Form12(utilisateur1.score);
                    form12.FormClosed += (s, args) =>
                    {
                        WinRestart();

                    }; form12.Show();

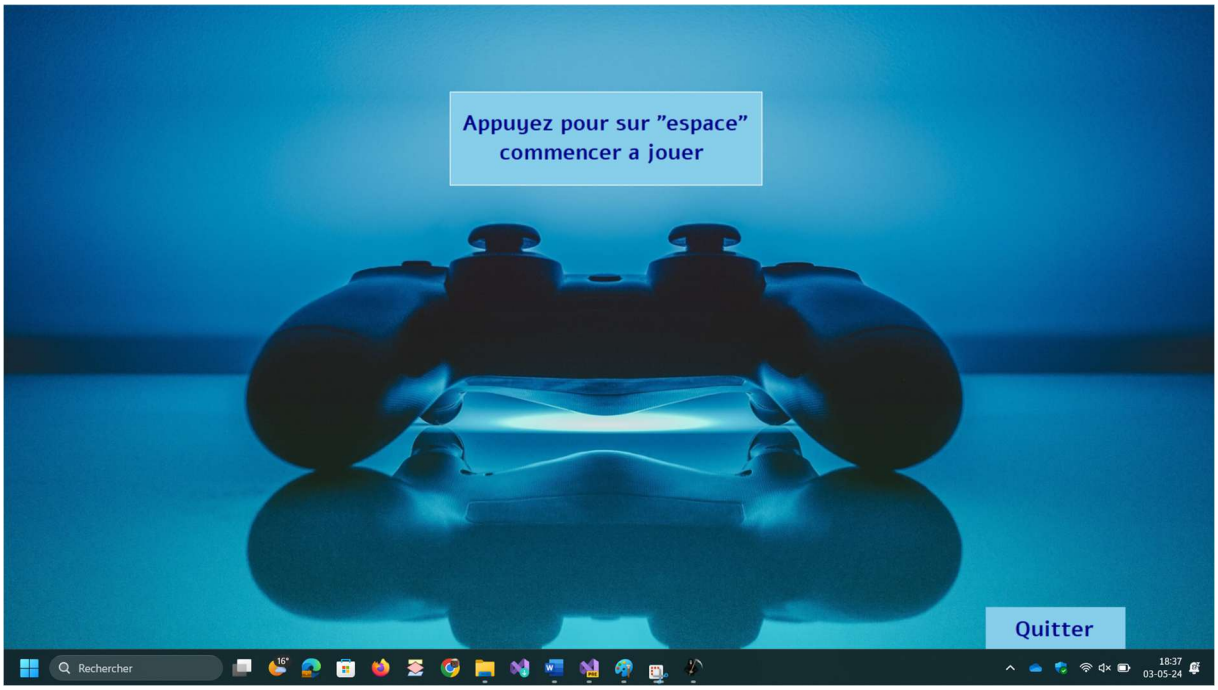
                }
                if (utilisateur1.score > 35 && utilisateur1.score < 45 || utilisateur1.score == 45)
                {
                    Form13 form13 = new Form13(utilisateur1.score);
                    form13.FormClosed += (s, args) =>
                    {
                        WinRestart();
                    };
                    form13.Show();
                }
                if (utilisateur1.score > 45 && utilisateur1.score < 53)
                {
                    Form14 form14 = new Form14(utilisateur1.score);
                    form14.FormClosed += (s, args) =>
                    {
                        WinRestart();
                    };
                    form14.Show();
                }
                if (utilisateur1.score == 53)
                {
                    Form15 form15 = new Form15(utilisateur1.score);
                    form15.FormClosed += (s, args) =>
                    {
                        WinRestart();
                    };
                    form15.Show();
                }

            }
        }
```

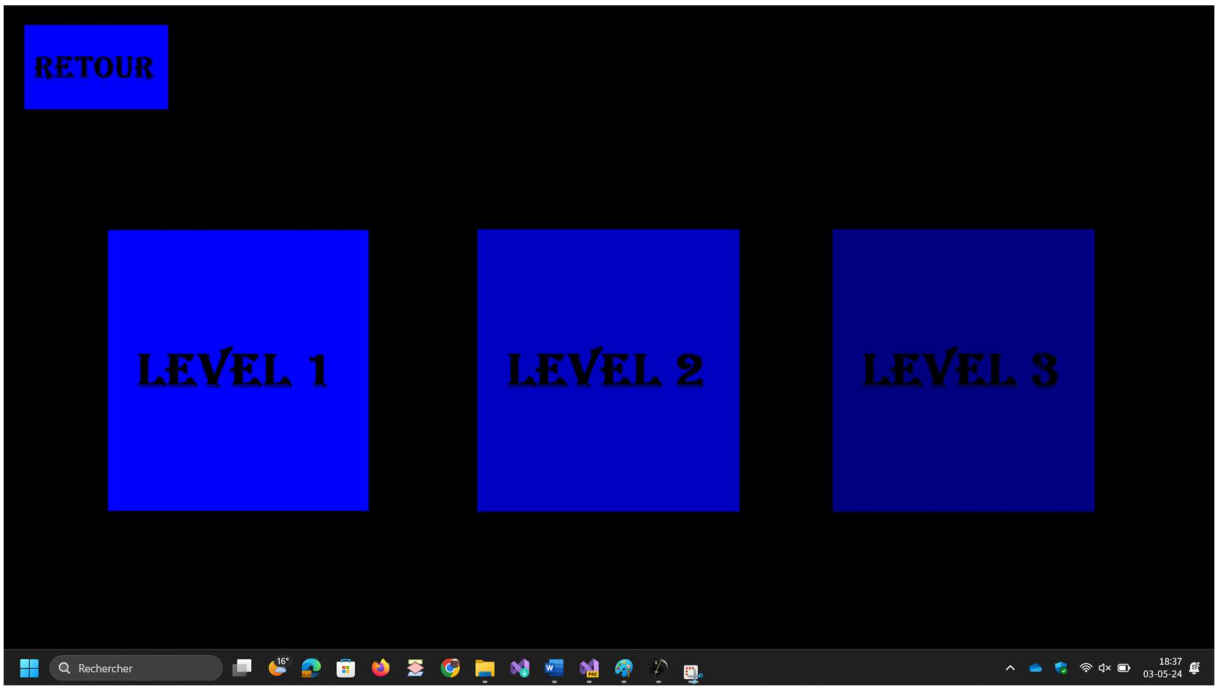
Ici dans le Form5, c’est le même code que le 1 mais il y a plus de pièces, c’est tous.

Pour terminer, les Forms 2 et 3.

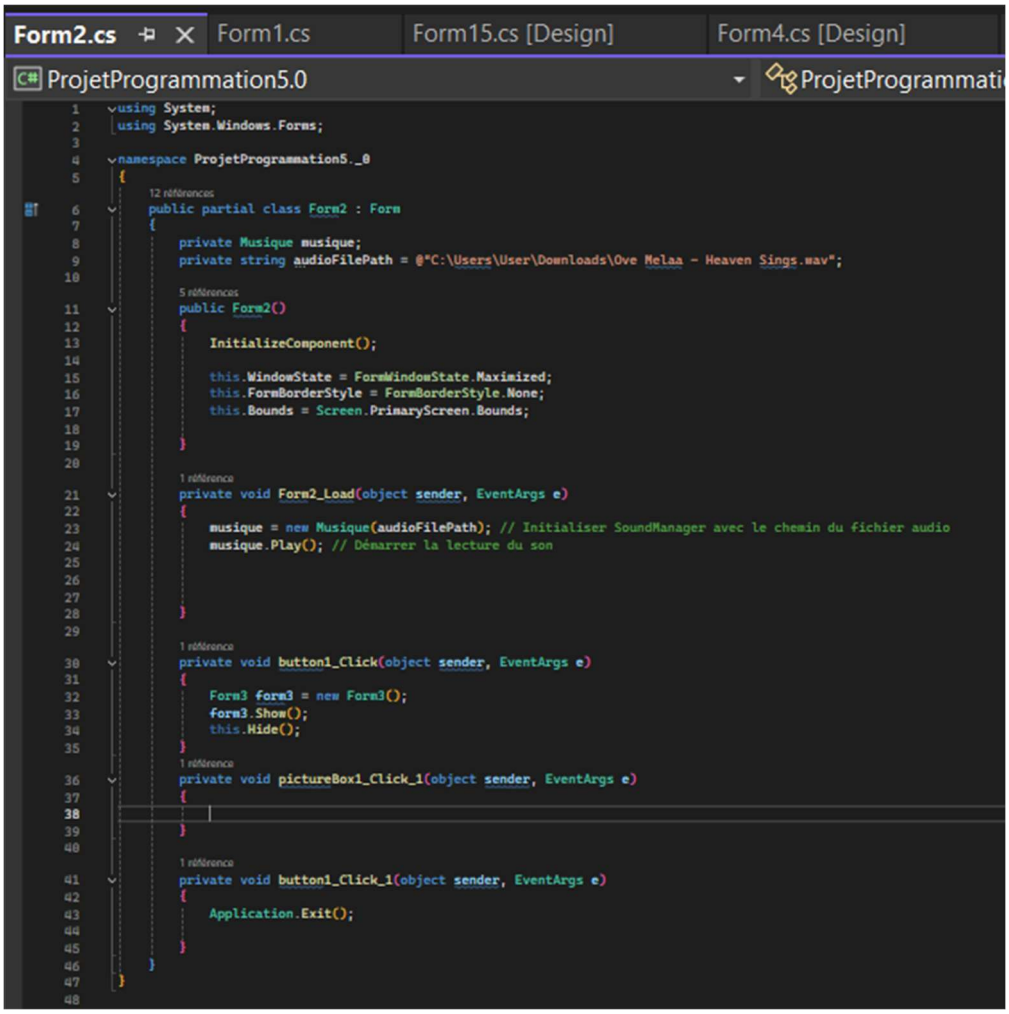
Form2 interface :



Interface Form3 :



Code Form 2 :



La musique se joue quand on lance la partie et lorsqu'on appuyé sur un bouton, on arrive au form3 et un bouton pour quitter.




```
namespace ProjetProgrammation5._0
{
    11 références
    public partial class Form3 : Form
    {
        private Form1 form1Instance;
        private Form4 form4Instance;


        4 références
        public Form3()
        {
            InitializeComponent();

            this.WindowState = FormWindowState.Maximized;
            this.FormBorderStyle = FormBorderStyle.None;
            this.Bounds = Screen.PrimaryScreen.Bounds;

            form1Instance = new Form1();
            form4Instance = new Form4();

        }

        0 références
        private void button2_Click(object sender, EventArgs e)
        {

        }

        1 référence
        private void button5_Click(object sender, EventArgs e)
        {
            Form2 form2 = new Form2();
            form2.Show();
            this.Hide();
        }

        1 référence
        private void button1_Click(object sender, EventArgs e)
        {
            if (!form1Instance.Visible)
            {
                form1Instance.Show();
            }

            // Fermez ce formulaire (Form3)
            this.Close();
        }

        1 référence
        private void button2_Click_1(object sender, EventArgs e)
        {
            if (!form4Instance.Visible)
            {
                form4Instance.Show();
            }

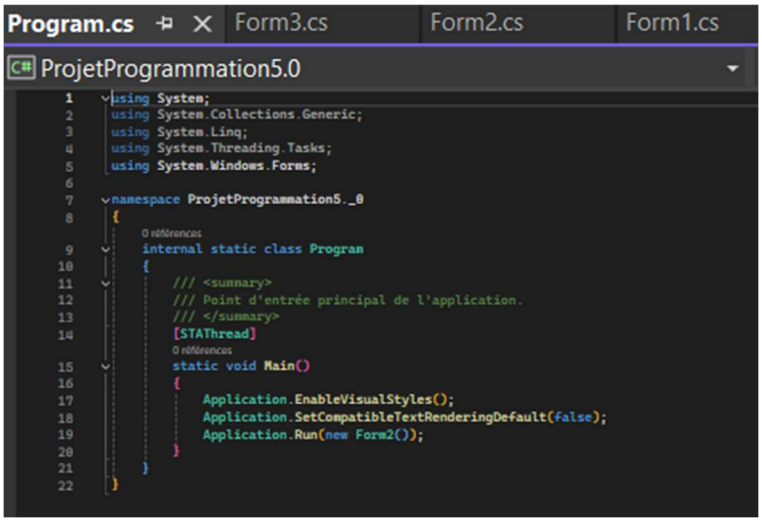
            // Fermez ce formulaire (Form3)
            this.Close();
        }

        1 référence
        private void button3_Click(object sender, EventArgs e)
        {
            Form5 form5 = new Form5();
            form5.Show();
            this.Hide();
        }

        1 référence
        private void Form3_Load(object sender, EventArgs e)
        {

        }
```

Ici pareille plusieurs boutons qui nous envoie au différents Form de jeux et un autre qui nous renvoie au menu principal (Form2). Et ici sont gérer les Form, donc si il y a plus de 1 Form alors faut fermer le précédents.



Et pour terminer le program.cs, évidement il n'y a rien.

Temps : Je n'ai pas compté mais je dirais facilement 40, 50 h.

J'ai commencé dimanche et on est vendredi.

Source :

NAudio nuget package documentation :

https://learn.microsoft.com/fr-fr/nuget/quickstart/install-and-use-a-package-in-visual-studio

Objet pixel :

https://www.udemy.com/course/learn-c-sharp-by-making-a-flappy-bird-game-with-moo-ict/?referralCode=E88C233BA830A367AF32&couponCode=LEADERSALE24Ahttps://www.youtube.com/watch?v=yw_sn9DonH4Pixel :

Piexel :

https://opengameart.org/latest

https://www.canva.com/design/DAGEF1-9vsI/NmWJD4w6Fq6jNYCAJG4bCA/edit (Celui la je l'ai fait moi meme)